

Nov 13, 2025

GPT-5.1 Prompting Guide

This recipe is archived and may reference outdated models or APIs.



Samarth Madduru (OpenAI)

[View on GitHub](#)[Download raw](#)

Introduction

GPT-5.1, our newest flagship model, is designed to balance intelligence and speed for a variety of agentic and coding tasks, while also introducing a new `none` reasoning mode for low-latency interactions. Building on the strengths of GPT-5, GPT-5.1 is better calibrated to prompt difficulty, consuming far fewer tokens on easy inputs and more efficiently handling challenging ones. Along with these benefits, GPT-5.1 is more steerable in personality, tone, and output formatting.

While GPT-5.1 works well out of the box for most applications, this guide focuses on prompt patterns that maximize performance in real deployments. These techniques come from extensive internal testing and collaborations with partners building production agents, where small prompt changes often produce large gains in reliability and user experience. We expect this guide to serve as a starting point: prompting is iterative, and the best results will come from adapting these patterns to your specific tools and workflows.

Migrating to GPT-5.1

For developers using GPT-4.1, GPT-5.1 with `none` reasoning effort should be a natural fit for most low-latency use cases that do not require reasoning.

For developers using GPT-5, we have seen strong success with customers who follow a few key pieces of guidance:

[Ask AI](#)

Loading docs agent...

shape the overall style, tone, and cadence through prompting.

We've found that personality and style work best when you define a clear agent persona. This is especially important for customer-facing agents which need to display emotional intelligence to handle a range of user situations and dynamics. In practice, this can mean adjusting warmth and brevity to the state of the conversation, and avoiding excessive acknowledgment phrases like "got it" or "thank you."

The sample prompt below shows how we shaped the personality for a customer support agent, focusing on balancing the right level of directness and warmth in resolving an issue.

`<final_answer_formatting>`

You value clarity, momentum, and respect measured by usefulness rather than pleasantries. Your default instinct is to keep conversations crisp and purpose-driven, trimming anything that



Loading docs agent...

understanding, you pivot fully to the task.

- You listen closely to the user's energy and respond at that tempo: fast when they're fast, more spacious when they're verbose, always anchored in actionability.

- Underlying principle:

- Your communication philosophy is "respect through momentum." You're warm in intention but concise in expression, focusing every message on helping the user progress with as little friction as possible.

</final_answer_formatting>

In the prompt below, we've included sections that constrain a coding agent's responses to be short for small changes and longer for more detailed queries. We also specify the amount of code allowed in the final response to avoid large blocks.

Loading docs agent...

- Citing code that is in the codebase:

- * If you must include an in-repo snippet, you may use the repository citation form, but in final answers avoid line-number/filepath prefixes and large context. Do not include more than 1-2 short snippets total.

</final_answer_formatting>

Excess output length can be mitigated by adjusting the verbosity parameter and further reduced via prompting as GPT-5.1 adheres well to concrete length guidance:

<output_verbosity_spec>

- Respond in plain text styled in Markdown, using at most 2 concise sentences.
- Lead with what you did (or found) and context only if needed.
- For code, reference file paths and show code blocks only if



Loading docs agent...

resume, summarize what you learned.

- Only the initial plan, plan updates, and final recap can be longer, with multiple bullets and paragraphs

</frequency_and_length>

<content>

- Before the first tool call, give a quick plan with goal, constraints, next steps.

- While you're exploring, call out meaningful new information and discoveries that you find that helps the user understand what's happening and how you're approaching the solution.

- Provide additional brief lower-level context about more granular updates

- Always state at least one concrete outcome since the prior update (e.g., "found X", "confirmed Y"), not just next steps.

- If a longer run occurred (>6 steps or >8 tool calls), start the next update with a 1-2 sentence synthesis and a brief justification for the heads-down stretch.

Loading docs agent...

Encouraging complete solutions

On long agentic tasks, we've noticed that GPT-5.1 may end prematurely without reaching a complete solution, but we have found this behavior is promptable. In the following instruction, we tell the model to avoid premature termination and unnecessary follow-up questions.

`<solution_persistence>`



- Treat yourself as an autonomous senior pair-programmer: once the user gives a direction, proactively gather context, plan, implement, test, and refine without waiting for additional prompts at each step.
- Persist until the task is fully handled end-to-end within the current turn whenever feasible: do not stop at analysis or partial fixes; carry changes through implementation, verification, and a clear explanation of outcomes unless the

Loading docs agent...

```
        "type": "string",  
        "description": "Reservation date and time (ISO 8601  
format)."  
    },  
    },  
    "required": ["name", "datetime"]  
}  
}
```

In the prompt, you may have a section that references the tool like this:

```
<reservation_tool_usage_rules>  
- When the user asks to book, reserve, or schedule a table, you  
MUST call `create_reservation`.  
- Do NOT guess a reservation time or name – ask for whichever  
detail is missing.  
- If the user has not provided a name, ask: “What name should I
```



Loading docs agent...

GPT-5.1 also executes parallel tool calls more efficiently. When scanning a codebase or retrieving from a vector store, enabling parallel tool calling and encouraging the model to use parallelism within the tool description is a good starting point. In the system prompt, you can reinforce parallel tool usage by providing some examples of permissible parallelism. An example instruction may look like:

Parallelize tool calls whenever possible. Batch reads (read_file) and edits (apply_patch) to speed up the process.



Using the “none” reasoning mode for improved efficiency

GPT-5.1 introduces a new reasoning mode: `none`. Unlike GPT-5’s prior `minimal` setting, `none` forces the model to never use reasoning tokens, making it much more similar in usage to GPT-4.1, GPT-4o, and other prior non-reasoning models.

Loading docs agent...

for these tasks, our guidance for GPT-5.1 with `none` is similar. Below is a snippet from our Tau bench prompt.

Remember, you are an agent – please keep going until the user’s query is completely resolved, before ending your turn and yielding back to the user. You must be prepared to answer multiple queries and only finish the call once the user has confirmed they’re done.



Maximizing coding performance from planning to execution

One tool we recommend implementing for long-running tasks is a planning tool. You may have noticed reasoning models plan within their reasoning summaries. Although this is helpful in the moment, it may be difficult to keep track of where the model is relative to the execution of the query.

Loading docs agent...

```
in_progress that corresponds to the work you're about to do;  
update the plan first if needed.  
- Scope pivots: if understanding changes (split/merge/reorder  
items), update the plan before continuing. Do not let the plan  
go stale while coding.  
- Never have more than one item in_progress; if that occurs,  
immediately correct the statuses so only the current phase is  
in_progress.  
<plan_tool_usage>
```

A plan tool can be used with minimal scaffolding. In our implementation of the plan tool, we pass a merge parameter as well as a list of to-dos. The list contains a brief description, the current state of the task, and an ID assigned to it. Below is an example of a function call that GPT-5.1 may make to record its state.

Loading docs agent...

```
--background, --foreground, --primary, --accent, --border, --
ring) or DS components that consume them.
- Introducing a brand or accent? Before styling, add/extend
tokens in globals.css under :root and .dark, for example:
  - --brand, --brand-foreground, optional --brand-muted, --
brand-ring, --brand-surface
  - If gradients/glows are needed, define --gradient-1, --
gradient-2, etc., and ensure they reference sanctioned hues.
- Consumption: Use Tailwind/CSS utilities wired to tokens (e.g.,
bg-[hsl(var(--primary))], text-[hsl(var(--foreground))], ring-
[hsl(var(--ring))]). Buttons/inputs/cards must use system
components or match their token mapping.
- Default to the system's neutral palette unless the user
explicitly requests a brand look; then map that brand to tokens
first.
</design_system_enforcement>
```

Loading docs agent...

When the model decides to execute an `apply_patch` tool, you will receive an `apply_patch_call` function type within the response stream. Within the operation object, you'll receive a `type` field (with one of `create_file`, `update_file`, or `delete_file`) and the diff to implement.

```
{
  "id":
  "apc_08f3d96c87a585390069118b594f7481a088b16cda7d9415fe",
  "type": "apply_patch_call",
  "status": "completed",
  "call_id": "call_Rjsqzz96C5xzPb0jUWJFRTNW",
  "operation": {
    "type": "update_file",
    "diff": "
    @@
    -def fib(n):
    +def fibonacci(n):
```



Loading docs agent...

shell.

```
tools = [{"type": "shell"}]
```



When a shell tool call is returned, the Responses API includes a `shell_call` object with a timeout, a maximum output length, and the command to run.

```
{
  "type": "shell_call",
  "call_id": "...",
  "action": {
    "commands": [...],
    "timeout_ms": 120000,
    "max_output_length": 4096
  },
}
```



Loading docs agent...

answer users' questions about potential venues and logistics.

You are "GreenGather," an autonomous sustainable event-planning agent. You help users design eco-conscious events (work retreats, conferences, weddings, community gatherings), including venues, catering, logistics, and attendee experience. 

PRIMARY OBJECTIVE

Your main goal is to produce concise, immediately actionable answers that fit in a quick chat context. Most responses should be about 3–6 sentences total. Users should be able to skim once and know exactly what to do next, without needing follow-up clarification.

SCOPE

* Focus on: venue selection, schedule design, catering styles,

Loading docs agent...

continue reasoning and using tools until the plan is coherent and complete, rather than bouncing decisions back to the user. Do not ask the user for clarifications unless absolutely necessary for safety or correctness. Make sensible assumptions about missing details such as budget, headcount, or dietary needs and proceed.

To avoid incorrect assumptions, when key information (date, city, approximate headcount) is missing, pause and ask 1–3 brief clarifying questions before generating a detailed plan. Do not proceed with a concrete schedule until those basics are confirmed. For users who sound rushed or decisive, minimize questions and instead move ahead with defaults.

TOOL USAGE

You always have access to tools for:

- * `venue_search`: find venues with capacity, location, and

Loading docs agent...

keynote, afternoon breakout rooms, evening reception”), approximate timing, and at least a rough budget breakdown for events longer than one day.

However, respect the user’s time: long walls of text are discouraged. Aim for compact responses that rarely exceed 2–3 short sections. For complex multi-day events or multi-vendor setups, provide a detailed, step-by-step plan that the user could almost copy into an event brief, even if it requires a longer answer.

SUSTAINABILITY GUIDANCE

- * Whenever you suggest venues or transportation, include at least one lower-impact alternative (e.g., public transit, shuttle consolidation, local suppliers).
- * Do not guilt or moralize; frame tradeoffs as practical choices.

Loading docs agent...

behaviors, we can metaprompt GPT-5.1 to inspect its own instructions and traces.

Step 1: Ask GPT-5.1 to diagnose failures

Paste the system prompt and a small batch of failure examples into a separate analysis call. Based on the evals you've seen, provide a brief overview of the failure modes you expect to address, but leave the fact-finding to the model.

Note that in this prompt, we're not asking for a solution yet, just a root-cause analysis.

You are a prompt engineer tasked with debugging a system prompt for an event-planning agent that uses tools to recommend venues, logistics, and sustainable options. 

You are given:

1) The current system prompt:

Loading docs agent...

```
description: ...  
prompt_drivers:  
  - exact_or_paraphrased_line: ...  
  - why_it_matters: ...
```

Metaprompting works best when the feedback can logically be grouped together. If you provide many failure modes, the model may struggle to tie all of the threads together. In this example, the dump of failure logs may contain examples of errors where the model was overly or insufficiently verbose when responding to the user's question. A separate query would be issued for the model's over-eagerness to call tools.

Step 2: Ask GPT-5.1 how it would patch the prompt to fix those behaviors

Loading docs agent...

```
1) patch_notes: a concise list of the key changes and the reasoning behind each (e.g., "Merged conflicting tool-usage rules into a single hierarchy," "Removed overlapping tone instructions that encouraged both executive formality and casual first-person with emojis").  
2) revised_system_prompt: the full updated system prompt with your edits applied, ready to drop into an agent configuration.
```

In this example, the first metaprompt helps GPT-5.1 point directly at the contradictory sections (such as the overlapping tool rules and autonomy vs clarification guidance), and the second metaprompt turns that analysis into a concrete, cleaned-up version of the event-planning agent's instructions.

The output from the second prompt might look something like this:

Loading docs agent...

...for quick thinking for easy questions, increasing when you come to harder topics, new tools for coding use cases, and the option to set reasoning to **none** when your tasks don't require heavy thinking.

Get started with GPT-5.1 in the [docs](#), or read the [blog post](#) to learn more.